WEEK 2 · DAY 1

Enterprise LLM Architecture

Tokens, Prompt Architecture, Grounding & Enterprise Safety

1-Hour Lecture Session — Concepts & Architecture (No Live Platform Demo)

Rajesh Pasham

Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and architecture lecture — no live Palantir platform work today
Session Format	One combined live session for India & Europe
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3+)	Build a Copilot grounded in your batch's Week 1 Ontology — each batch's own use case (Claims, Underwriting, Fraud/SIU, or Policy Servicing)

Where Today Fits

Week 1 built the Ontology. Week 2 starts putting AI on top of it.

Every Object Type, Marking, and security policy your batch built in Week 1 is what today's concepts get applied to. A Copilot is only as good — and only as safe — as the architecture decisions underneath it. Today is entirely about those decisions, before you build anything in Day 3.

Today's Roadmap

Five topic groups, building toward one architecture

1	**Enterprise LLM Architecture**	What's actually different about running LLMs at enterprise scale
2	**Model Selection**	Choosing a model deliberately, not by default
3	**Prompt Architecture**	System, user, and contextual instructions — and how to structure them
4	**Context Engineering & Grounding**	Getting the right data in front of the model, safely
5	**Safety, Routing & Security**	Hallucination, ambiguity, routing, and enterprise risk

By the End of Today

The architecture decisions every Copilot in this program depends on

Explain what changes at enterprise scale

Token economics, context limits, and model behavior under real production load

Structure a prompt using system, user, and contextual layers

Not one undifferentiated block of instructions

Ground a model in Ontology data responsibly

The difference between plausible and true, and why that distinction is architectural

Reason about hallucination, ambiguity, and routing

As design problems to architect around, not surprises to react to

TOPIC 1

Enterprise LLM Architecture

What's actually different about running LLMs at enterprise scale

What Makes an Architecture “Enterprise”

It's not the model — it's everything around it

Many users, many use cases, one governed foundation

A single Ontology and governance model has to serve every Copilot built on top of it, not just one prototype

Every answer has to be accountable

Who asked, what was retrieved, what was said — all auditable, not just “the model said so”

Cost and latency are architecture decisions, not afterthoughts

The model you'd pick for a demo is often the wrong one to run at production volume

Tokens & Context Windows

The unit everything else in this session is measured in

Token	The unit a model reads and generates in — roughly a word-fragment, not a word or a character
Context Window	The maximum number of tokens a model can consider at once — system prompt, conversation history, and retrieved context all share this budget
Why This Matters Architecturally	Retrieving “more context” isn't free — it competes for space with everything else, and can push out what actually matters

Model Limitations to Design Around

Every model has boundaries — good architecture assumes them, not ignores them

Knowledge cutoffs

A model's training data has a boundary in time — it doesn't know what happened after, and won't tell you that reliably on its own

Latency under real load

A response that's fast in a demo can behave differently under concurrent enterprise traffic

Consistency, not certainty

The same prompt can produce different phrasing on different runs — architecture should tolerate that, not assume determinism

TOPIC 2

Model Selection

Choosing a model deliberately, not by default

Model-Selection Criteria

The questions that should come before "which model is best"

Task Complexity	A simple summarization task rarely needs the largest, most expensive model available
Latency Requirements	An interactive Copilot has a different tolerance than a nightly batch process
Data Sensitivity	What the model will see determines whether a Palantir-hosted model is sufficient, or a registered/bring-your-own model is required
Cost at Scale	The per-call cost that looked negligible in testing can dominate a budget at production call volume

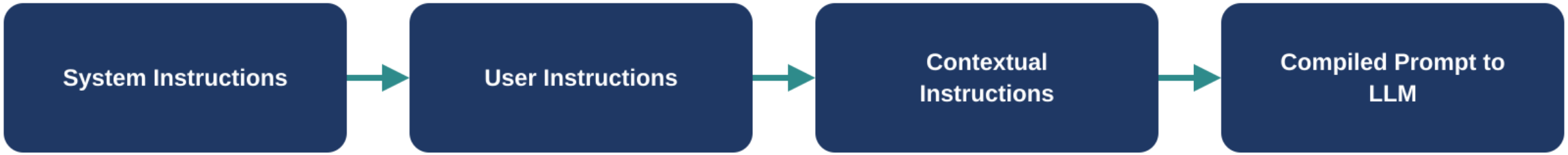
TOPIC 3

Prompt Architecture

System, user, and contextual instructions — and how to structure them

Three Layers of Instruction

Each layer answers a different question for the model



System	Who is this application, and what are its permanent rules? Set once, rarely changes per call.
User	What is this specific person asking right now?
Contextual	What retrieved data or application state is relevant to answering it?

System Instructions

The layer that defines the application's boundaries

Lead with the overview

What is this Copilot for, in one or two sentences — before any detail

State constraints explicitly, not implicitly

“Summarize and flag” is different from “recommend an approval” — say which one, don't assume it's obvious

This is where “never do X” belongs

Boundaries stated once here apply to every user interaction, not re-stated per question

User & Contextual Instructions

The layers that change on every call

User Instructions	The literal question or request — often exactly what the person typed, sometimes lightly normalized
Contextual Instructions	Retrieved Ontology objects, document excerpts, or application state — assembled per call, not hardcoded
The Architectural Point	System instructions are designed once; user and contextual instructions are designed to vary — confusing the two is a common source of inconsistent Copilot behavior

Zero-Shot vs. Few-Shot Techniques

How much example is worth including — and the cost of including it

Zero-Shot

No examples in the prompt — just instructions. Cheapest in tokens; relies entirely on the model's general capability.

Few-Shot

A small number of worked examples included in the prompt. Improves consistency for tricky formats, at a real token cost.

Rule of thumb: start zero-shot. Add examples only once you've seen a specific, repeatable failure mode that examples would fix — not preemptively.

Structured Output

Making a model's response something your system can actually use

Free text is for people; structured output is for systems

If a Function or Action needs to consume the model's answer, ask for a defined shape — not prose to parse later

Constrain the response format explicitly

Specify field names and types in the prompt or output schema, don't hope the model infers your intent

Validate anyway

A model can still return malformed output under a structured-output request — downstream validation is still your responsibility

Prompt Templates

Treating a prompt's structure as a reusable asset

Why Template	The same five-part structure (Overview, Context, Tools, Constraints, Output) should be reusable across every Copilot your batch builds, not reinvented each time
What Stays Fixed	The section structure and the boundaries/constraints pattern
What Varies	The specific data, tools, and domain language for each use case — this is where your batch's Claims, Underwriting, Fraud, or Servicing specifics go

TOPIC 4

Context Engineering & Grounding

Getting the right data in front of the model, safely

Context Engineering at Enterprise Scale

The same discipline from Week 1, now feeding a model instead of a person

Context is retrieved, not pasted

What the model sees should come from a governed retrieval step — not a static block of text someone wrote once

Governance travels with the data

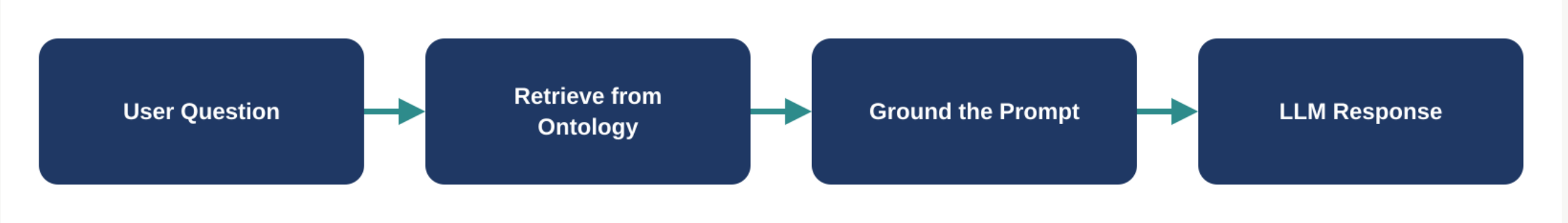
If your Week 1 Marking protects a field, that protection has to hold when an LLM retrieves it too — not just when a person views it

Less, well-chosen context beats more, unfiltered context

A shorter, relevant context window produces better answers than a longer, noisy one

Grounding LLMs with Ontology Data

Answering from your governed model, not the model's general knowledge



Grounding means the model's answer is built from retrieved, governed Ontology objects — your batch's Claim, Application, Fraud Case, or Policy records — not from what the model "remembers" generally about insurance.

Retrieval-Augmented Generation (RAG)

The general pattern grounding is built on

The Core Idea	Retrieve relevant data first, then include it in the prompt — rather than relying on the model's training data alone
Standard RAG	Typically retrieves unstructured text chunks from documents
Ontology-Grounded Retrieval	Retrieves governed, structured Ontology objects — anchored in your Week 1 model, not just similar-sounding text. Most production Copilots use both, combined.

TOPIC 5

Safety, Routing & Security

Hallucination, ambiguity, routing, and enterprise risk

Model-Routing Patterns

Not every question should go to the same model

Route by task complexity

A simple lookup doesn't need the same model as a multi-step reasoning task — route accordingly to control cost

Route by data sensitivity

A question touching highly sensitive fields may need to route to a registered model, not a default Palantir-hosted one

Route by confidence

A low-confidence response can route to a different model — or to a human — rather than being returned as-is

Handling Hallucination

Designing for a known failure mode, not hoping it doesn't happen

What It Is	A model producing a fluent, confident answer that isn't actually supported by the retrieved data
The Architectural Response	Ground every claim in retrieved context, and require the model to cite what it retrieved — don't just ask it to “be accurate”
The Honest Limit	No prompt eliminates hallucination completely — this is exactly why Week 6's Evals exist, to measure it rather than assume it away

Handling Ambiguity

When the question itself doesn't have one clear answer

Ambiguity isn't always a bug to fix

Some questions genuinely have more than one reasonable interpretation — the model should say so, not silently pick one

Design an explicit escalation path

A Copilot that isn't confident should say so and route to a human — exactly the escalation pattern in this week's Practical Lab

Enterprise Security Considerations

Everything from Week 1 still applies — an LLM doesn't get a pass

Roles and Markings still gate what's retrieved

An LLM querying the Ontology is still a query — your Week 1 governance model applies to it exactly as it applies to a person

Prompt injection is a real, specific risk

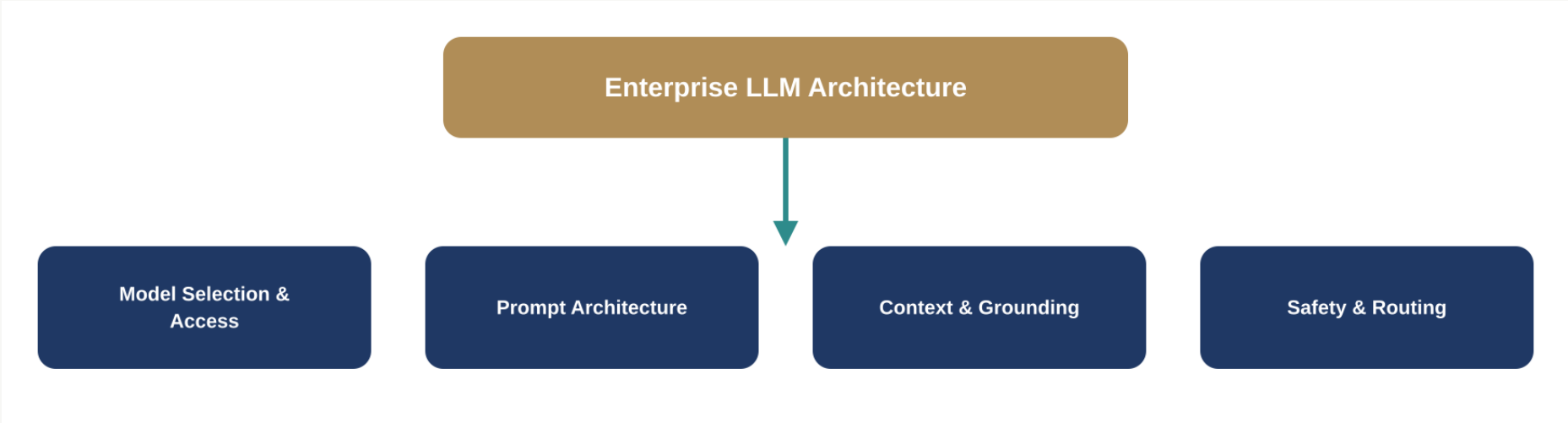
Retrieved content (a document, a user question) can contain text trying to override your system instructions — design defensively

Every model call should be auditable

Which model, what was retrieved, what was returned — traceable, not a black box

Putting It Together

Today's five topic groups as one architecture



Every Copilot your batch builds from Day 3 onward is an instance of this same architecture — a model chosen deliberately, a prompt structured in layers, context grounded in your governed Ontology, and safety patterns designed in from the start, not bolted on after.

Common Pitfalls

Patterns worth avoiding before Day 3's build

Picking a model before defining the task	Model choice should follow task complexity, latency needs, and data sensitivity — not the other way around
Mixing system and contextual instructions	Permanent rules and per-call data belong in different layers — blurring them produces inconsistent behavior
Retrieving more context “to be safe”	More context competes for the same window as everything else — targeted retrieval beats broad retrieval
Treating hallucination as a prompt-wording problem	It's an architecture problem — grounding, citation, and Evals address it; clever wording alone doesn't

Recap & What's Next

What We Covered Today

- Enterprise LLM architecture: tokens, context windows, limitations
- Model-selection criteria
- Prompt architecture: system, user, contextual layers
- Zero-shot/few-shot, structured output, prompt templates
- Context engineering, Ontology grounding, RAG
- Model routing, hallucination, ambiguity, security

Day 2 — LLM Pipelines & AI FDE Workflows

- Designing reusable LLM pipelines
- Ontology-aware retrieval, function/tool integration
- The AI Forward Deployed Engineering workflow
- From rapid prototype to operational Copilot
- Also a concepts lecture — no live platform work

Questions

Rajesh Pasham